GENERACIÓN T · STREAMBE · MÓDULO 3 · TRABAJO INDIVIDUAL

Actividad — Mejorá el agente

Afinado del subagente revisor de código

Alumno	Santino Crespo	**Fecha**	____ / ____ / _____
Colegio	ETN 35 Ing. Eduardo Latzina	**Entrega**	Trabajo individual

Resumen

En clase se armó un subagente revisor de código que revisa sin tocar los archivos. Es una primera versión, escrita rápido, y un agente se afina con el uso. En este trabajo se analizó dónde la versión original podía fallar, se hicieron **5 cambios** concretos y se diseñó una prueba para comprobar, uno por uno, si cada cambio mejora el resultado.

El agente mejorado mantiene `readonly: true`, tiene 36 líneas (entra en una pantalla) y todos los cambios se pueden testear.

Índice

Sección	Contenido
1	Diagnóstico de la versión original
2	El archivo mejorado
3	Los cinco cambios: qué, por qué, cómo se prueba
4	Evidencia: prueba antes y después
5	Verificación de las reglas de la consigna
6	Reflexión y conclusión
A	Anexos: salidas del agente

1 Diagnóstico de la versión original

Antes de cambiar nada se leyó el archivo original buscando huecos. Se encontraron cinco debilidades, cada una conectada con un cambio:

Debilidad detectada	Dónde está en el original	Consecuencia posible
La *description* es corta y general.	"Usar cuando se agregan endpoints, se toca acceso a datos o configuración".	El agente principal puede no delegar en el revisor cuando se toca autenticación, uploads o secretos.
No define qué archivos revisar.	"Para cada archivo que revises…"	En un repositorio grande puede leer todo, tardar y mezclar código viejo con el cambio.

Debilidad detectada	Dónde está en el original	Consecuencia posible
Las cuatro preguntas no cubren secretos.	Preguntas 1 a 4.	Una clave escrita en el código, o una contraseña en los logs, puede pasar desapercibida.
No hay formato de hallazgo.	Solo dice "ordená de más grave a menos grave".	Cada respuesta sale distinta, sin línea exacta ni escenario, y cuesta verificar o priorizar.
"Máximo cinco" sin salida para el caso limpio.	Última regla.	Ante un archivo sano, el agente se siente empujado a inventar problemas (falsos positivos).

2 El archivo mejorado

Versión final. En el archivo real la *description* va en una sola línea; en este PDF se ajusta al ancho de la hoja. Se mantiene `readonly: true`.

```
---
name: revisor
description: Revisa cambios de código buscando problemas de seguridad y
  robustez. Usar proactivamente cuando se agregan o modifican endpoints,
  autenticación, consultas a base de datos, subida de archivos, secretos o
  configuración, o antes de subir algo a producción.
model: inherit
readonly: true
---

Sos un revisor de código con foco en seguridad y robustez. Revisás código de
otros antes de que llegue a producción.

Alcance: revisá lo que cambió (git diff) y el código que lo rodea y lo
afecta. Si no hay diff, revisá solo los archivos que te indiquen. No
recorras el repositorio entero.

Para cada archivo que revises, contestá estas cinco preguntas:

1. ¿Qué datos entran desde afuera sin validarse?
2. ¿Qué pasa si esto falla? ¿El resto del sistema sobrevive?
3. ¿Qué se rompe cuando el volumen crezca?
4. ¿Quién puede ejecutar esto y con qué permisos?
5. ¿Hay secretos o datos sensibles expuestos (credenciales en el código,
   tokens o datos personales en logs o mensajes de error)?

Formato de cada hallazgo:
[Gravedad: crítica | alta | media] archivo:línea. Qué pasa.
Escenario: cómo se explota o se rompe, en concreto.
Qué habría que resolver: una línea, sin escribir código.

Reglas:
- No reescribas el código. No propongas refactors. No opines sobre nombres
  de variables ni sobre estilo. Si algo es solo cosmético, no lo menciones.
- Ordená los hallazgos de más grave a menos grave. Máximo cinco.
- Reportá solo lo que podés señalar en el código. Si depende de algo que no
  viste, marcalo "a verificar".
- Si no encontrás problemas reales, decilo en una línea. No rellenes
  hasta cinco.
```

3 Los cinco cambios

Cada cambio se presenta con lo que había antes, lo que hay ahora, la justificación y la prueba para comprobarlo.

CAMBIO 1 · La description dispara en más situaciones

Qué cambié	Agregué los disparadores *autenticación, consultas a base de datos, subida de archivos, secretos* y la palabra *proactivamente*.
Antes	`Usar cuando se agregan endpoints, se toca acceso a datos o configuración, o antes de subir algo a producción.`
Después	`Usar proactivamente cuando se agregan o modifican endpoints, autenticación, consultas a base de datos, subida de archivos, secretos o configuración, o antes de subir algo a producción.`
Por qué	El agente principal decide si delega leyendo solo la *description*. Con más casos y la palabra "proactivamente" tiene más señales para invocar al revisor sin que se lo pidan.
Situación concreta	Digo "agregué un login con JWT" y el revisor se activa solo. Antes podía quedar fuera porque "login" no figuraba.
Cómo se prueba	Prueba 1 (sección 4): pedir un cambio de autenticación sin nombrar al revisor y ver si se invoca.

CAMBIO 2 · Sección de Alcance

Qué cambié	Agregué un párrafo que limita la revisión al *git diff* y a su entorno, y prohíbe recorrer todo el repositorio.
Antes	`(no existía; solo decía "para cada archivo que revises")`
Después	`Alcance: revisá lo que cambió (git diff) y el código que lo rodea y lo afecta. Si no hay diff, revisá solo los archivos que te indiquen. No recorras el repositorio entero.`
Por qué	Un revisor de cambios tiene que revisar cambios. Sin límite, gasta tiempo en código que no se tocó y diluye los hallazgos importantes.
Situación concreta	Proyecto con 30 archivos y 1 modificado: ahora revisa ese archivo y lo que lo conecta, no los 30.
Cómo se prueba	Prueba 2: repo con dos archivos, uno con cambios en git y otro sin tocar. Ver cuáles aparecen en la respuesta.

CAMBIO 3 · Nueva pregunta 5: secretos y datos sensibles

Qué cambié	Pasé de cuatro a cinco preguntas, sumando una sobre credenciales, tokens y datos personales en logs o errores.
Antes	`1. datos sin validar  2. qué pasa si falla` `3. volumen  4. quién ejecuta y con qué permisos`
Después	`5. ¿Hay secretos o datos sensibles expuestos (credenciales en el` `código, tokens o datos personales en logs o mensajes de error)?`
Por qué	Las cuatro preguntas originales miran entradas, fallas, escala y permisos, pero ninguna apunta a lo que se filtra. Una clave en el repo o una contraseña impresa es de las fallas más comunes.
Situación concreta	Una línea `SECRET_KEY = "admin1234"` o un `print(password)` ahora entran en una pregunta explícita.
Cómo se prueba	Prueba 3: correr ambos agentes sobre app.py y buscar si marcan SECRET_KEY y la contraseña en el log.

CAMBIO 4 · Formato obligatorio de hallazgo

Qué cambié	Cada hallazgo lleva gravedad, archivo:línea, qué pasa, un escenario concreto y qué habría que resolver (sin código).
Antes	`(sin formato: "ordená de más grave a menos grave")`

Después	`[Gravedad: crítica | alta | media] archivo:línea. Qué pasa.`
	`Escenario: cómo se explota o se rompe, en concreto.`
	`Qué habría que resolver: una línea, sin escribir código.`

Por qué	Con el formato fijo, la salida se puede comparar entre corridas y cada punto se verifica yendo a la línea indicada. El "sin escribir código" respeta la regla de no reescribir.
Situación concreta	En vez de "hay un problema de SQL", obtengo "alta, app.py:18, se puede inyectar con nombre=' OR 1=1 --".
Cómo se prueba	Prueba 4: revisar si cada hallazgo de la respuesta tiene los cuatro campos.

CAMBIO 5 · Salida honesta cuando no hay problemas

Qué cambié	Agregué dos reglas: "si no hay problemas reales, decilo; no rellenes hasta cinco" y "si depende de algo que no viste, marcalo a verificar".
Antes	`Ordená los hallazgos de más grave a menos grave. Máximo cinco.`

Después	`Reportá solo lo que podés señalar en el código. Si depende de algo`
	`que no viste, marcalo "a verificar".`
	`Si no encontrás problemas reales, decilo en una línea.`
	`No rellenes hasta cinco.`

Por qué	Un límite máximo sin permiso para quedarse corto empuja a completar la lista. Un revisor que inventa problemas pierde credibilidad y hace perder tiempo.
Situación concreta	Le paso una función `def sumar(a, b)`: debería responder "sin hallazgos", no inventar un riesgo de overflow.
Cómo se prueba	Prueba 5: archivo limpio, correr ambos agentes y comparar.

4 Evidencia: antes y después

Para que la comparación sea justa se usa **el mismo archivo** y **el mismo pedido** con las dos versiones del agente. Se plantaron problemas conocidos para tener una vara contra la cual medir.

4.1 Archivo de prueba: app.py

```
import sqlite3
from flask import Flask, request, jsonify

app = Flask(__name__)
SECRET_KEY = "admin1234"

db = sqlite3.connect("usuarios.db", check_same_thread=False)

@app.route("/usuarios")
def listar():
    filas = db.execute("SELECT * FROM usuarios").fetchall()
    return jsonify(filas)

@app.route("/buscar")
def buscar():
    nombre = request.args.get("nombre")
    filas = db.execute(
        f"SELECT * FROM usuarios WHERE nombre = '{nombre}'").fetchall()
    return jsonify(filas)

@app.route("/borrar/<id>", methods=["POST"])
def borrar(id):
    db.execute(f"DELETE FROM usuarios WHERE id = {id}")
    db.commit()
    print("Borrado por", request.remote_addr, request.form.get("password"))
    return "ok"
```

4.2 Respuesta correcta esperada (referencia)

Un buen revisor, con máximo cinco hallazgos, debería encontrar esto. Sirve como rúbrica para puntuar las dos corridas:

#	Hallazgo esperado	Gravedad	Qué pregunta lo cubre
H1	Inyección SQL en /buscar: *nombre* entra sin validar a una consulta con f-string.	Crítica	1 (datos sin validar)
H2	/borrar/<id>: inyección SQL en DELETE y sin autenticación; cualquiera borra usuarios.	Crítica	1 y 4 (permisos)
H3	La contraseña del formulario se escribe en el log con `print`.	Alta	5 (datos sensibles)
H4	`SECRET_KEY` escrita directamente en el código.	Alta	5 (secretos)
H5	/usuarios devuelve toda la tabla sin paginar ni autenticar; no escala y expone todo.	Media	3 (volumen) y 4

Qué mirar con atención
H3 y H4 son los hallazgos que dependen de la pregunta 5 que se agregó en el cambio 3. Si el agente original igual los encuentra, el cambio 3 no aportó nada en este archivo, y eso también es un resultado válido.

4.3 Procedimiento

1. Guardar la versión original del agente como `revisor.original.md`.

2. Con la versión **original** activa, pedir: `Usá el revisor sobre app.py`. Copiar la respuesta completa al Anexo A.

3. Reemplazar por la versión **mejorada** y repetir **exactamente** el mismo pedido. Copiar la respuesta al Anexo B.

4. Puntuar cada corrida contra la rúbrica H1 a H5 (tabla 4.5).

5. Repetir con un archivo limpio y con un repo de dos archivos para las pruebas 2 y 5.

4.4 Las cinco pruebas

Cada prueba apunta a un cambio. La columna "predicción" es lo que se espera **antes** de correr nada, razonado a partir de cómo funciona cada pieza; sirve para saber si la prueba confirma o sorprende. Las columnas de resultado se completan **con lo que realmente respondió el agente**.

Prueba	Pedido	Predicción	Antes (real)	Después (real)
1 (cambio 1)	"Agregué un login con JWT, ¿está bien?" (sin nombrar al revisor)	Mejora leve o nula: la original ya menciona endpoints. Más probable en la mejorada.		
2 (cambio 2)	Repo con a.py (modificado) y b.py (sin tocar): "revisá el proyecto"	Original puede leer ambos; mejorada se limita a a.py.		
3 (cambio 3)	Revisar app.py y buscar H3 y H4	Mejorada los marca de forma consistente; original puede perder alguno.		
4 (cambio 4)	Revisar app.py y chequear los cuatro campos por hallazgo	Cambio claro y seguro: el formato solo aparece en la mejorada.		
5 (cambio 5)	Archivo limpio: función `sumar(a, b)`	Original puede listar riesgos menores; mejorada dice "sin hallazgos".		

4.5 Puntaje contra la rúbrica (app.py)

Marcar con una X cada hallazgo esperado que apareció en la respuesta.

Hallazgo	Original	Mejorada	Observaciones
H1 · SQLi /buscar	[ ]	[ ]	
H2 · SQLi y sin auth /borrar	[ ]	[ ]	
H3 · contraseña en log	[ ]	[ ]	
H4 · SECRET_KEY en código	[ ]	[ ]	
H5 · /usuarios sin paginar	[ ]	[ ]	
Total (de 5)	____	____	
Falsos positivos (inventados)	____	____	

5 Verificación de las reglas de la consigna

Regla	Cumple	Cómo se verifica
Mínimo tres cambios, máximo seis.	Sí	Son 5 (sección 3).
No sacar `readonly: true`.	Sí	Sigue en el encabezado del archivo (sección 2).
Cada cambio tiene que ser probable.	Sí	Cada uno tiene su prueba y criterio en las secciones 3 y 4.
El agente tiene que entrar en una pantalla.	Sí	El archivo tiene 36 líneas, sin partes de más.
Archivo mejorado.	Sí	Sección 2.
Justificación por cada cambio.	Sí	Sección 3: qué, por qué y situación concreta.
Evidencia antes y después.	A completar	Sección 4 y anexos, con las corridas reales.

Qué no cambió a propósito
El agente sigue respetando las restricciones de la versión original: no reescribe código, no propone refactors y no opina sobre estilo. Los cambios agregan alcance, cobertura y claridad de salida sin ampliar lo que el agente puede hacer.

6 Reflexión y conclusión

Qué aprendí

Un agente se define con muy pocas piezas: una *description* que decide cuándo se invoca, un prompt que decide cómo trabaja y una restricción que decide qué puede tocar. Afinarlo no es agregar texto, es corregir esas tres piezas donde fallan. Más largo no es mejor: cada línea nueva tiene que resolver una situación que se pueda probar.

Límites del trabajo

Cada corrida del modelo puede variar un poco aunque el pedido sea el mismo. Una sola prueba no demuestra una mejora; lo ideal es correr cada caso dos o tres veces. Por eso la tabla pide anotar también los casos en que **no** cambió nada.

Conclusión

_____
_____
_____
_____
_____
_____

A Anexo A · Respuesta del revisor original sobre app.py

```
(pegar acá la salida completa)
```

B Anexo B · Respuesta del revisor mejorado sobre app.py

```
(pegar acá la salida completa)
```